Universitat de Lleida

High Performance Computing
Convolution Operation OpenMP Practice

Students:
Joel Castillo Estallo
David Carreras Castañ

Date: 19 April 2026

Main decisions taken to parallelize the problem

The goal of this practice was to parallelize the convolution operation using OpenMP, starting from the provided sequential C implementation. The original program reads a PPM image, reads a kernel matrix, processes the image in partitions if needed, applies the convolution independently to the three color channels (R, G and B), and stores the resulting image.
The assignment requires analyzing the sequential code, identifying the parts susceptible to parallelization, and justifying the selected strategy while taking into account important things such as dependencies, memory traffic, computational cost and synchronization needs.
After analyzing the serial code, we observed that the main computational hotspot is the convolution phase. In the main() function, the convolution is applied separately to the red, green and blue channels through three calls to convolve2D(). In contrast, the other phases of the code are mainly related to file reading, file writing, kernel loading and memory copying.
These phases are important for the total execution time, but they involve I/O operations or relatively simple memory operations and therefore they are not the best starting point for OpenMP parallelization. The most expensive and computation-dense part is clearly the nested loop structure inside convolve2D().
The first important implementation decision was to rewrite the original sequential convolution kernel before adding OpenMP directives.

The provided serial convolve2D() function used moving pointers such as inPtr, inPtr2, outPtr and kPtr, whose values evolved across loop iterations. Although that implementation is valid in serial code, it carries iteration state and makes the outer loop harder to parallelize safely. For this reason, we replaced the pointer-based traversal with an index-based version, where each output pixel is computed explicitly from indices (i, j) and neighboring positions (ii, jj). This modification preserves the functionality of the original algorithm but makes each output row easier to compute independently. It also simplifies reasoning about data dependences, which is essential for OpenMP.

Once the convolution kernel had been rewritten, the selected OpenMP strategy was to parallelize the outer loop over image rows inside convolve2D() using *#pragma omp parallel for schedule(static).* This loop was chosen because it provides a natural work decomposition: each iteration computes one output row, and different rows write to disjoint memory locations in the output array. The input image and the kernel are only read, so there are no write-write conflicts between threads. In addition, each row contains enough work to amortize the OpenMP overhead, since for each pixel the algorithm must traverse the whole kernel. This makes row-level decomposition more suitable than parallelizing the innermost loops, whose iterations are much smaller and would likely generate a worse overhead-to-work ratio.

The OpenMP version also follows the timing requirement of the assignment. The original code used *gettimeofday(),* but for the OpenMP delivery the statement explicitly requires the use of *omp_get_wtime().* Therefore, the timing code in *main()* was adapted so that the measured phases (reading image, copying image structure, reading kernel matrix, convolution, writing the resulting image and total elapsed time) use *omp_get_wtime()*. This keeps the same phase breakdown as in the original code while satisfying the requirement of the project.
The source code can be found in the github repository:
https://github.com/JoelCastEst/HPC-OMP-Project

Alternatives considered

Several alternatives were considered before selecting the final implementation.

The first alternative was to parallelize **only the three channel convolutions**, that is, executing the R, G and B convolutions in parallel while keeping the convolution kernel itself sequential. This approach is possible because the three channels are independent and write to different output buffers. However, it was not selected as the main strategy because it only exposes on a three-task schedule (R, G and B). This limits scalability and does not exploit the full amount of data parallelism available inside each channel. On a node with more than three useful worker threads, the parallelism of this strategy is inherently limited. In addition, the computationally dominant work is still inside the convolution loops, so a finer decomposition is preferable.

The second alternative considered was to **combine RGB-level parallelism with the parallelization of the internal row loop**. In other words, one possibility was to launch the three convolve2D() calls in parallel and at the same time keep the OpenMP parallel for inside convolve2D(). This option was discarded because it introduces nested parallelism, which increases implementation complexity and makes the runtime behavior harder to control. Depending on the OpenMP runtime configuration, nested regions may either serialize the inner level or create too many threads, increasing overhead and potentially harming performance. Since the assignment also values implementation clarity and justified decisions, we preferred a single-level OpenMP strategy that is easier to reason about and evaluate.

Therefore, the final choice was to parallelize only the outer loop inside convolve2D(). This solution provides a fine-grain work decomposition, low synchronization needs, direct mapping to the shared-memory node model, and a clear implementation that remains close to the original sequential structure.

Scalability of the Program

To evaluate the scalability of the OpenMP implementation, we measured the execution time of the convolution program for the image and kernel combinations required by the assignment. The statement specifies five test images (im01 to im05) and four kernels (5x5 Sharpen, 25x25 Random, 49x49 Random, and 99x99 Random). It also specifies that im05 must be executed with at least three partitions because of its size. Using these combinations, we obtained serial execution times and OpenMP execution times with 2 and 4 threads. From these values, the speedup and efficiency metrics were calculated.
The standard formulas used were:

$$Speedup = \frac{T_{serial}}{T_{parallel}} \qquad Efficiency = \frac{Speedup}{p}$$

Where T_{serial} is the execution time of the serial version, $T_{parallel}$ is the execution time of the OpenMP version, and p is the number of threads used in the parallel execution. In our executions, p was 2 or 4 depending on the configuration (as OMP only operates up to 4 cores).
Tables 1 to 5 show the measured execution times together with the corresponding speedup and efficiency values for each image and each kernel. In order to keep the report concise and readable, we chose to preserve the exact values in tables and also provide two summary plots: one overall plot for speedup and one overall plot for efficiency.

Image	**Kernel**			
Im01	**kernel5x5_Sharpen.txt**	**Kernel25x25_Random2.txt**	**Kernel49x49_Random2txt**	**Kernel99x99-Random2.txt**
Serial	0.112099	0.641137	2.197561	8.308373
OMP 2 threads	0.092219	0.650759	2.084144	7.625461
OMP 4 Threads	0.077999	0.352565	1.145758	4.280987
2 threads speedup	1.216	0.985	1.054	1.090
2 threads efficiency	0.608	0.493	0.527	0.545
4 threads speedup	1.437	1.818	1.918	1.941
4 threads efficiency	0.359	0.455	0.479	0.485

Table 1. Table with arguments to calculate the elapsed times of the Im01.

Image	Kernel			
Im02	**kernel5x5_Sharpen.txt**	**Kernel25x25_Random2.txt**	**Kernel49x49_Random2txt**	**Kernel99x99-Random2.txt**
Serial	0.192128	1.621497	5.636850	22.308050
OMP 2 threads	0.191297	1.502437	5.372465	20.228907
OMP 4 Threads	0.161151	0.871742	2.917048	11.133542
2 threads speedup	1.004	1.079	1.049	1.103
2 threads efficiency	0.502	0.540	0.525	0.551
4 threads speedup	1.192	1.860	1.932	2.004
4 threads efficiency	0.298	0.465	0.483	0.501

Table 2. Table with arguments to calculate the elapsed times of the Im02.

Image	Kernel			
Im03	**kernel5x5_Sharpen.txt**	**Kernel25x25_Random2.txt**	**Kernel49x49_Random2txt**	**Kernel99x99-Random2.txt**
Serial	0.436407	3.317847	11.481928	46.686213
OMP 2 threads	0.590642	3.092492	10.853094	41.526319
OMP 4 Threads	0.323869	1.729308	5.867542	22.389592
2 threads speedup	0.739	1.073	1.058	1.124
2 threads efficiency	0.369	0.536	0.529	0.562
4 threads Speedup	1.347	1.919	1.957	2.085
4 threads efficiency	0.337	0.480	0.489	0.521

Table 3. Table with arguments to calculate the elapsed times of the Im03.

Image Im04	Kernel			
	kernel5x5_Sharpen.txt	Kernel25x25_Random2.txt	Kernel49x49_Random2txt	Kernel99x99-Random2.txt
Serial	17.955136	162.369510	581.173065	2529.892722
OMP 2 threads	17.547366	150.182225	549.250044	2185.992003
OMP 4 Threads	14.464880	84.165687	289.594108	1139.728952
2 threads speedup	1.023	1.081	1.058	1.157
2 threads efficiency	0.512	0.541	0.529	0.579
4 threads Speedup	1.241	1.929	2.007	2.220
4 threads efficiency	0.310	0.482	0.502	0.555

Table 4. Tble with arguments to calculate the elapsed times of the Im04.

Image Im05	Kernel			
	kernel5x5_Sharpen.txt	Kernel25x25_Random2.txt	Kernel49x49_Random2txt	Kernel99x99-Random2.txt
Serial	65.472774	643.877559	2340.317637	10373.402927
OMP 2 threads	61.516055	612.465364	2216.543357	8988.523680
OMP 4 Threads	50.160328	341.993654	1145.793739	4499.576473
2 threads speedup	1.064	1.051	1.056	1.154
2 threads efficiency	0.532	0.526	0.528	0.577
4 threads Speedup	1.305	1.883	2.043	2.305
4 threads efficiency	0.326	0.471	0.511	0.576

Table 5. Table with arguments to calculate the elapsed times of the Im05.

As said before, we decided to represent the values in a table, as we consider, tables capture the exact information that plots sometimes can't. Even so, we decided to plot two overall graphics, for Speedup [Image 1] and for Efficiency [Image 2]

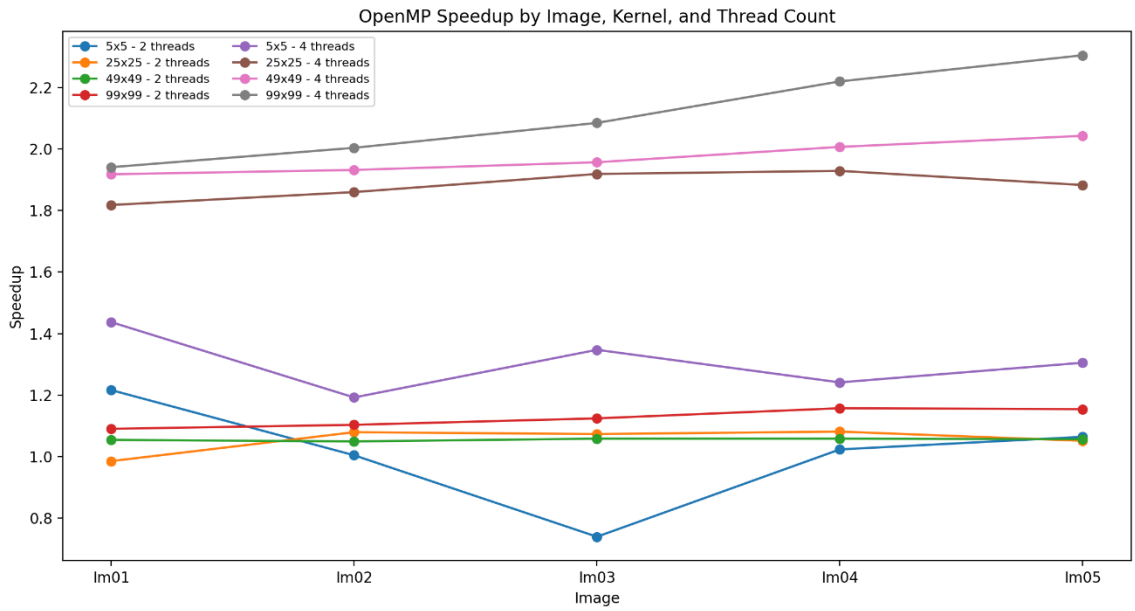

Image 1. Graph for overall Speedup by Image, Kernel and Thread Count

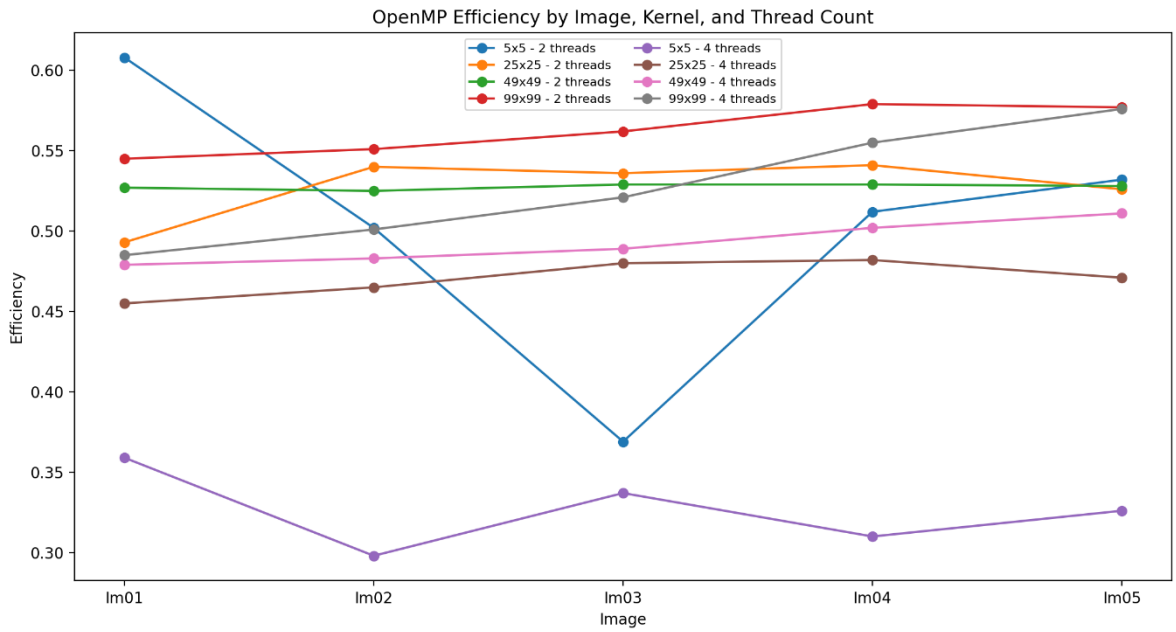

Image 2. Graph for overall Efficiency by Image, Kernel and Thread Count

Overall, the best speedup values are obtained with 4 threads and with the largest kernels, especially the 99x99 kernel. For example, for im05 and the 99x99 kernel, the speedup reaches 2.305 with 4 threads, while the corresponding efficiency is 0.576. Similar behavior can be observed for im04 with the same kernel, where the speedup with 4 threads reaches 2.220 and the efficiency reaches 0.555. In contrast, the smallest image and kernel combinations show much lower gains, and in one case (im03 with the 5x5 kernel and 2 threads) the speedup is even below 1, meaning that the parallel version is slightly slower than the serial one.

These results indicate that the OpenMP implementation scales better when the computational workload is large enough to compensate for the parallel overhead. This trend is consistent across the test cases provided: the larger the kernel and the larger the image, the better the speedup obtained with 4 threads. The efficiency values also confirm this trend, since the largest kernels tend to maintain more stable efficiency values than the smallest ones.

Scalability of the Program

The obtained results can be explained by considering the structure of the convolution algorithm, the selected OpenMP strategy and the effects of computational cost, memory traffic and synchronization.

In the convolution process, each output pixel is computed by combining the values of neighboring input pixels weighted by the selected kernel. Therefore, the computational cost per output pixel depends strongly on the kernel size. A 5x5 kernel performs much less work than a 99x99 kernel, while a larger image increases the number of pixels to be processed. As a consequence, larger kernels and larger images generate a significantly higher total workload, which makes parallelization more profitable. This is exactly what can be observed in the measured results, where the largest kernels achieve the best speedup values, especially with 4 threads.

The selected row-level parallelization shows a clear dependency pattern: there are almost no true dependences between rows of the output image. Each thread computes different rows, writes to disjoint positions of the output array and only reads from the input image and the kernel. Because of this, synchronization requirements are minimal. No reductions, locks or critical sections are needed in the main computational kernel. This is one of the reasons why the chosen strategy is effective: it exploits natural data decomposition with very low synchronization overhead.

However, the results also show that parallel speedup is limited for the smallest cases. This is particularly visible with the 5x5 kernel and the smaller images. In these executions, the total amount of work is too small compared with the OpenMP overhead associated with thread creation, scheduling and coordination. In addition, the complete program includes phases that remain sequential, such as file reading, kernel reading, copying structures and writing the output image.

Since these phases are outside the main parallel region, they limit the maximum achievable speedup. For this reason, the global speedup is always lower than the ideal linear speedup, and for the smallest cases it may even become worse than serial execution. The case of im03 with the 5x5 kernel and 2 threads, where the speedup is 0.739, is a good example of this behavior.

Memory traffic is another relevant factor. Even though the row-level decomposition avoids synchronization problems, the convolution algorithm still performs a large number of memory accesses to the input image. As kernel size grows, each output pixel requires more neighbor accesses, which increases the arithmetic cost but also increases memory traffic. In this practice, the larger kernels still benefit more from OpenMP, which suggests that the increase in useful computation is enough to amortize the memory-access overhead and the OpenMP runtime overhead. For the smallest kernels, however, the relative cost of memory access and OpenMP management is more significant compared with the amount of arithmetic work, which reduces the effectiveness of parallelization.

The efficiency results support the same interpretation. For 2 threads, efficiency values are often around 0.50–0.58 for medium and large kernels, which indicates a reasonable use of the two threads. For 4 threads, efficiency decreases in most cases, which is expected because speedup does not grow proportionally with the number of threads. Nevertheless, for the largest kernels the 4-thread efficiency remains acceptable, reaching values around 0.55–0.58 in the best cases. This means that the selected strategy scales moderately well within the node, especially for the largest workloads, even if it is still far from ideal scalability.

In summary, the main reasons for the observed behavior are the following: first, the convolution phase is the dominant computational part of the application and is therefore the best target for OpenMP; second, row-level decomposition provides independent tasks with low synchronization needs; third, larger kernels and images improve speedup because they offer enough useful work to compensate for parallel overhead; and fourth, the remaining sequential phases and the cost of memory accesses prevent ideal scalability, especially in the smallest configurations. These observations are consistent with both the structure of the code and the measurements obtained in the experiments.

Bibliography

1. **OpenMP Architecture Review Board.** *OpenMP API Specification – omp_get_wtime.* Official OpenMP specification page describing the wall-clock timing routine used in the OpenMP version. https://www.openmp.org/spec-html/5.0/openmpsu160.html
2. **OpenMP Architecture Review Board.** OpenMP API Specification (HTML version). Official OpenMP reference for directives, clauses, and shared-memory parallel programming concepts. https://www.openmp.org/spec-html/5.1/openmp.html